SQL Sentinel SQL Injection Detection Project Technical Report

CNS Project 76 | Streamlit implementation | 11 October 2026

This report explains how we built SQL Sentinel and how its algorithms, training pipeline, interface, evaluation, source files, and deployment work. The current app compares six regular-expression rules with character TF-IDF plus Logistic Regression. The original Naive Bayes implementation remains as a legacy baseline. It is not the model used by the Streamlit ML tab.

On our authored synthetic test, ML correctly classifies 370 of 384 inputs, versus 228 for rules. This supports an ML advantage on this corpus only. The application is an educational text detector, not a production database firewall or proof that ML is universally superior.

1 Purpose and scope

SQL injection occurs when untrusted input becomes executable query syntax rather than remaining a data value. If a program concatenates a username into a quoted condition, input such as admin' OR 1=1 -- may close the quote, insert an always-true condition, and comment out the remaining query. Success depends on the actual query, database dialect, escaping, and permissions.

Our detector classifies strings. It does not connect to a database, construct a SQL query, execute payloads, intercept traffic, or verify exploitation. Suspicious means a detector flagged the text. Not flagged means no positive prediction, not guaranteed safety. Actual applications should use prepared statements and appropriate database permissions.

The project lets students see rule matches, inspect learned feature contributions, and compare methods using one shared input. The three tabs are Rule-based, ML-based, and Comparison. Detailed explanations belong in this report so the UI can remain concise.

2 How we built the project

We first created normalization, six rules, a character n-gram Naive Bayes baseline, a small synthetic corpus, an evaluator, and a standard-library HTTP app. We then simplified the browser interface into three tabs and added processing animations. These original files are retained for reference.

The original corpus favored configured signatures. For the current app, we authored more varied benign SQL-related prose and attack-like patterns, separated training, validation, and test families, and fitted a scikit-learn Pipeline. We selected regularization with validation F1, saved the pipeline, evaluated test predictions, and built a Streamlit interface around the artifact. Ten automated checks pass locally.

Both corpus and model changed. The new results cannot isolate the benefit of replacing Naive Bayes alone. A controlled model comparison would train both methods on the same frozen corpus with the same selection protocol. We did not tune the decision threshold against final test scores.

3 Architecture and data flow

Offline flow: training_data.py or an external CSV feeds train_model.py. The trainer validates rows, fits candidates, selects a pipeline, and produces models/sql_detector.joblib, data/training_corpus.csv, and results/trained_metrics.json.

Online flow: a sidebar widget supplies input to analyze. The callback reads current session state, normalizes the input, evaluates both detectors, and stores results. Tab content renders real rule checks, feature contributions, a score, and comparison metrics. CSS in an iframe illustrates processing. It does not run the classifier.

The browser displays Streamlit widgets; Python performs inference on the server. Local inference runs on the user's computer; deployed inference runs on the Cloud server. There is no paid API, LLM, SQL execution engine, or persistent database. ROOT = Path(__file__).resolve().parent makes artifact paths independent of the launch directory.

Saving the vectorizer and classifier together keeps feature columns aligned with their coefficients. A classifier without its fitted vocabulary and IDF values would not be sufficient to reproduce predictions.

4 Technologies and design choices

Component	Choice	Purpose
Python	3.12.8 in training run	Shared training inference and UI logic
Streamlit	1.65.0	Interactive Python app
scikit-learn	1.9.1	Vectorizer Pipeline and classifier
joblib	1.6.0	Save and reload fitted artifact
pandas	3.0.6	Benchmark dataframe
Python re	Six regex signatures	Transparent baseline
CSV and JSON	Corpus and metrics	Reproduction and review
HTML and CSS	st.iframe	Visual explanatory animation
Streamlit Community Cloud	GitHub based deployment	Hosted application

Python allows the UI and trainer to share detector functions. Streamlit avoids maintaining a separate frontend API. Sparse linear classification is fast to train, economical on CPU, and explainable through weights. Dependency versions are pinned to the tested direct dependencies; platform verification remains necessary.

5 Corpus construction

training_data.py defines FAMILIES by split, label, and template. Label 0 represents benign template intent; label 1 represents attack-like intent. A family is one authored template, not an independent user session or attack campaign.

Split	Benign families	Attack families	Variants per family	Rows	Rows per class
Training	26	26	30	1560	780

Split	Benign families	Attack families	Variants per family	Rows	Rows per class
Validation	10	10	12	240	120
Test	16	16	12	384	192
Total	52	52	Varies	2184	1092

build replaces {x} with a neutral identifier cycling through alex, riley, guest, entry, search, and item, followed by the variant index. The same scheme is used in both classes, avoiding explicit label markers in identifiers. Variants remain highly correlated: changing an identifier does not create a new independent case.

Benign families include ordinary searches, quoted names, SQL lessons, Boolean-condition notes, and function documentation. Attack families include Boolean predicates, UNION, stacked commands, delay functions, existence subqueries, string comparisons, and extraction functions. Test templates differ from training templates, but SQL concepts recur.

No external real-world traffic or independent labels were collected. Legitimate SQL prose intentionally challenges context-insensitive rules. This design is useful for education but influences the measured comparison. It does not represent production attack prevalence or a representative deployment distribution.

CSV columns are text, label, split, and family. save uses csv.DictWriter, UTF-8, a header, and newline handling. External CSV loading uses utf-8-sig to accept an initial byte-order mark. Missing columns can raise KeyError; the loader does not provide a specialized schema-validation message for every malformed input.

6 Split validation and leakage controls

validate rejects an empty corpus, converts labels to integers, accepts only labels 0 and 1 and splits train, validation, and test, and checks nonblank strings of at most 4,000 characters. Every split must contain both labels.

A dictionary tracks each normalized string's split and label. A repeated normalized string is rejected if it crosses splits or has a conflicting label. Same-split duplicates with the same label are currently allowed. A second dictionary rejects a family appearing in different splits.

The validator does not infer semantic families or detect every near-duplicate. Differently named but similar templates can pass. Authors must group sensibly and avoid revising a test set to improve scores. These checks prevent direct cross-split duplication; they do not remove correlated variants or author expectations.

Only training rows fit the vocabulary, IDF values, and coefficients. Validation chooses a candidate but is not used for refitting. Test rows are evaluated after selection. Perfect validation scores on a small correlated set do not establish independent generalization.

7 Preprocessing details

normalize calls urllib.parse.unquote twice. %27 becomes an apostrophe; %2527 can be decoded through the second pass. Decoding stops after two passes. unquote does not translate plus signs into spaces as unquote_plus would.

It lowercases text, collapses whitespace runs into one space using re.sub, and strips outer whitespace. This reduces case and spacing variation. The original input remains in session state; classification uses its normalized representation.

The rule path additionally replaces closed block comments matching /*.*?*/ with a space. ML receives normalized text with comment characters retained. The app repeats rule comment removal when generating the checklist so displayed matches agree with rule_detect.

This is not a SQL tokenizer or parser. Comment removal can affect quoted text; not every dialect, escape, Unicode variant, encoding, or comment form is handled. Normalization does not reconstruct query context or determine whether the payload is executable.

8 Rule based algorithm

rule_detect normalizes, removes closed block comments, searches all six patterns with re.search, and collects matching names. Any match yields malicious=True. No matches yield False and the reason No configured rule matched. There is no training, severity weighting, score, or vote.

Rule	Pattern intent	Limit
Boolean condition	OR or AND followed by numeric or quoted equality	Does not calculate Boolean truth
UNION query	UNION optional ALL then SELECT	Can flag educational prose
Stacked command	Semicolon then DROP DELETE UPDATE INSERT ALTER EXEC	Incomplete command and dialect coverage
Delay function	SLEEP BENCHMARK PG_SLEEP calls or WAITFOR DELAY	Function documentation can be benign
Comment after quote	Quote then -- # or /*	Does not recognize every surrounding context
Metadata extraction	information_schema or load_file call	Documentation can contain the same tokens

Word boundaries constrain keyword matching. Noncapturing groups express alternatives, whitespace patterns allow spacing, and optional groups accept UNION SELECT or UNION ALL SELECT. Boolean numeric equality includes both 1=1 and 1=2: the rule detects a shape, not whether it is true. The quoted-equality expression is not a complete SQL string grammar.

The stacked rule expects the listed command after semicolon and optional whitespace. Delay rules expect a named function followed by an opening parenthesis, or WAITFOR DELAY. Metadata detection does not require a complete SELECT. The comment rule looks immediately after a quote, so a later comment marker is not automatically a match.

We use rules for transparency, determinism, speed, and a clear baseline. They miss patterns outside their signatures and cannot distinguish every tutorial from an attack. Expanding signatures can improve coverage while adding false positives. Six expressions are not an exhaustive SQL injection defense.

9 Character TF IDF representation

TfidfVectorizer converts normalized strings into overlapping character n-grams of lengths 2 through 5. For or 1, fragments include or, r-space, space-1, or-space, and or-space-1. Quotes, operators, comments, and spacing remain features.

Character features suit punctuation-rich short inputs and do not require a vocabulary of complete words. They can recognize related syntax across identifiers. They still do not parse SQL or understand exploit success.

For a positive frequency count, sublinear_tf=True uses TF = 1 + ln(count). Default smoothed IDF is ln((1 + N)/(1 + df)) + 1, where N is training documents and df is documents containing a feature. The raw TF-IDF value is TF multiplied by IDF. Default L2 normalization divides a row by its Euclidean length. Repeated substrings are moderated, and common substrings receive lower IDF.

Parameter	Value	Meaning
analyzer	char	Includes punctuation and spaces
ngram_range	2 to 5	Extracts overlapping short fragments
min_df	2	Requires occurrence in two training documents
max_features	20000	Vocabulary cap rather than target
sublinear_tf	True	Logarithmic positive frequency
use_idf	Default True	Applies document-frequency weighting
smooth_idf	Default True	Smooths the IDF calculation
norm	Default l2	Normalizes each document vector

The fitted vocabulary contains 4,054 features. Sparse matrices store nonzero entries, avoiding a dense array for every possible column. Inference ignores unknown n-grams. If an input has no known features, the intercept largely determines its score. Training-fitted IDF and vocabulary are reused unchanged on validation, test, and app input.

10 Logistic Regression algorithm

For feature vector x, Logistic Regression computes z = b + sum(w_j * x_j). b is the intercept and w_j is a coefficient. Its positive-class score is sigmoid(z) = 1 / (1 + exp(-z)). The app flags scores at least 0.5. Despite its name, this is binary classification rather than continuous-target regression.

Positive contributions push toward label 1; negative contributions push toward label 0. Training minimizes weighted logistic loss with regularization. Regularization discourages excessive coefficients; smaller C means stronger regularization. We test C=0.5, 2, and 8.

Setting	Value	Role
C	Selected 0.5	Inverse regularization strength
max_iter	1000	Optimizer iteration limit
class_weight	balanced	Accounts for class frequency
random_state	76	Recorded reproducibility setting
solver	Default lbfgs	Numerical optimizer

Setting	Value	Role
fit_intercept	Default True	Learns baseline logit
tol	Default 0.0001	Convergence tolerance
Effective penalty	L2	Constrains squared coefficient magnitude

In scikit-learn 1.9.1 the inspected l1_ratio default is 0.0, with the effective L2 configuration. The saved classifier reports 10 iterations. Balanced class weights are n_samples/(n_classes * class_count); both are effectively equal for this balanced corpus. The random_state setting does not mean every solver introduces randomness.

We chose Logistic Regression for efficient CPU inference, short training time, suitability for sparse text, and interpretable weights. TF-IDF supplies features and Logistic Regression supplies the decision: they are stages of one ML pipeline. It is not a neural network or LLM. Scores are uncalibrated and should not be read as the real-world probability of successful exploitation.

11 Training selection and persistence

train_model.py partitions rows by split and normalizes training and validation strings. Each Pipeline has named steps features and classifier. fit learns the vectorizer and classifier on training rows; validation predictions determine F1.

Candidate C	Validation F1	Outcome
0.5	1.0000	Selected
2.0	1.0000	Tied
8.0	1.0000	Tied

The selection condition is score > best_f1, so the first tied candidate wins. No cross-validation, threshold search, or final train-plus-validation refit occurs. The chosen model retains training-only vocabulary and weights.

joblib.dump writes models/sql_detector.joblib. TrainedDetector reloads it so evaluation and UI use the same saved pipeline. Aggregate test metrics and direct per-family correctness fractions are recorded. Each test family contains 12 variants; per-family reporting makes correlation visible without establishing independence.

The trainer writes data/training_corpus.csv and results/trained_metrics.json. The dataset SHA-256 is computed over JSON-serialized rows with sorted dictionary keys, not CSV bytes; row ordering matters. The recorded digest is c69d7c3615b7da2d8e262e5cc53edbaad7bce00163e60ec01a563c991a8c9ce7.

The recorded training_and_evaluation_seconds is about 1.62 seconds. It includes candidate fits, artifact save and load, aggregate evaluation, and family evaluation. It is not pure training time. Hardware and load change timings even when predictions remain identical.

12 Prediction and explanations

TrainedDetector.__init__ loads the trusted bundled joblib artifact. predict normalizes text and reads predict_proba([value])[0,1]. With labels 0 and 1, the second column is the attack-like class. The malicious Boolean uses the unrounded value; returned score is rounded to six decimals.

For explanations, the same text is transformed and converted to a sparse coordinate row. Each nonzero TF-IDF value is multiplied by its matching classifier coefficient. The function sorts signed contributions descending and returns six feature tokens and contributions, plus readable reasons.

These are the six largest signed values, not the largest absolute effects. Large negative evidence can be omitted. The intercept and other features also affect the decision, so adding displayed chips will not reproduce the score. Contributions are in logit units, not percentage-point changes. Overlapping fragments are correlated and are not independent causal explanations.

No known features can yield an empty chip list. Fetching vocabulary names on every prediction and separately computing probability and evidence adds overhead. The score is not calibrated; a displayed 86.3% is a model output rather than verified exploit likelihood.

13 Streamlit app lifecycle and state

set_page_config sets title, icon, wide layout, and expanded sidebar. The sidebar offers example selection, a textarea capped at 4,000 characters, an animation toggle, and a short hint. Login bypass is the initial payload. Other options include normal username, encoded attack, Boolean keyword, benign SQL lesson, and custom input.

model uses cache_resource to load the pipeline once per cache lifecycle for read-only inference. metrics uses cache_data for parsed JSON. Missing model causes a training message and stop. Missing or malformed metrics currently causes a runtime error rather than a custom fallback. Restart or clear caches after replacing artifacts.

session_state payload holds input. change_example replaces it and clears analysis. Editing the input also clears analysis. The analyze callback reads current session_state so it does not use a stale local string from a prior rerun. Blank input yields a warning.

All three Run buttons call analyze. It computes both detectors and all rule checks, then stores text, normalized, rule_based, ml_based, and checks under analysis. Both algorithms therefore run even when the user presses Run ML-based, enabling consistent results across tabs.

rule_based contains malicious and reasons. ml_based includes malicious, score, reasons, and features; each feature has token and contribution. checks contains name and matched for all six rules. State belongs to a browser session and is not persistent project storage.

The Rule-based tab shows checks; the ML-based tab shows learned features, score, threshold, and verdict. Comparison shows both verdicts, agreement, and a frozen benchmark table from JSON. Benchmark percentages are not generated from the current single input. The training-details expander records selection and limitations and offers JSON download.

14 Animation implementation

animation constructs trusted HTML CSS and embeds it with st.iframe. html.escape protects every user string, rule reason, and feature token included in markup. The kind is app-controlled and the numeric score is model-derived. Input is displayed as text rather than interpreted as HTML.

CSS grid lays out three nodes and two wires. Nodes pulse in sequence, packets move along wires, and evidence cards enter with opacity and vertical movement. A bar scales to the score; a fixed marker shows the 50% threshold. These are conceptual processing visuals based on actual results.

When enabled, evidence starts after roughly three seconds with small staggered delays; the verdict follows. There is no server sleep and these delays do not measure inference latency. Disabling animation or prefers-reduced-motion suppresses motion. Narrow-screen styles reduce padding and stack checks. content height lets the iframe size around its content.

All tab bodies execute on rerun, so hidden iframes can start animating before their tab opens. A fresh Run click retriggers analysis. The illustration is not a numerical trace of the optimizer or a replay of every coefficient calculation.

15 Worked examples

admin' OR 1=1 -- becomes admin' or 1=1 --. The Boolean rule matches. The comment-after-quote rule does not necessarily match because OR 1=1 intervenes between quote and comment. One match is sufficient. The trained model gives approximately 0.863 for this demonstration input, above 0.5; the app displays 86.3% and Suspicious input.

The encoded login example is decoded before classification, exposing punctuation and syntax. This demonstrates two-pass percent decoding but does not establish universal encoding coverage.

SQL textbook: UNION SELECT combines rows alex0 is benign educational text in our labeling. The UNION rule flags it. Learned surrounding fragments can help ML avoid some such alarms, although this reflects corpus cues rather than human understanding of intent. The complete false-positive and false-negative texts are stored in the benchmark JSON.

16 Evaluation definitions and edge cases

TP is an attack-labeled input flagged; TN is benign not flagged; FP is benign flagged; FN is an attack missed. evaluate calls each detector once per row, compares malicious with label, accumulates counts, and stores mismatched text, label, and prediction.

Accuracy = (TP + TN)/total. Precision = TP/(TP + FP). Recall = TP/(TP + FN). F1 = 2*precision*recall/(precision + recall). False-positive rate = FP/(FP + TN). Higher is better for the first four; lower is better for false-positive rate.

Zero precision and recall denominators yield zero; zero precision-plus-recall yields zero F1. Accuracy and false-positive rate require a nonempty set with benign examples, which aggregate test data supplies. Single-label family accuracy uses direct fractions to avoid unsafe denominators.

Accuracy alone can hide missed attacks or excessive alarms. Balanced test prevalence is not measured traffic prevalence. Precision can change with deployment class frequency even if class-conditional errors remain similar. No family-level confidence intervals are computed.

17 Final measured results

Outcome	Rule-based	ML-based
True positives	108	190
True negatives	120	180

Outcome	Rule-based	ML-based
False positives	72	12
False negatives	84	2
Correct predictions	228 of 384	370 of 384
Accuracy	59.38%	96.35%
Precision	60.00%	94.06%
Recall	56.25%	98.96%
F1	58.06%	96.45%
False-positive rate	37.50%	6.25%
Local ms per input	0.0059	1.6666

ML improves accuracy by about 36.98 percentage points on this corpus. It misses 2 attack-like rows instead of 84 and raises 12 false alarms instead of 72. Scores are measured rather than hard-coded verdicts. The dataset intentionally includes patterns outside the six configured signatures and SQL-related benign text.

Rules are faster. The timing is a single local pass and includes ML explanation work; it is not a controlled throughput benchmark. The original metrics.json uses different data and Naive Bayes, so it must not be mixed with current scores. trained_metrics.json is the authoritative current benchmark.

18 Source file and function guide

File	Responsibility
streamlit_app.py	Config widgets callbacks animation and comparison
trained_model.py	TrainedDetector init loads predict scores and explains
detector.py	normalize rule_detect grams and legacy NaiveBayes
training_data.py	FAMILIES build and CSV save
train_model.py	validate run and --csv command line
experiment.py	Shared evaluate and legacy run
dataset.py	Original small synthetic corpus
app.py	Legacy HTTP Handler and localhost server
web/index.html	Legacy browser interface
models/sql_detector.joblib	Current fitted Pipeline
results/trained_metrics.json	Current frozen test report
data/training_corpus.csv	Current rows and split labels
requirements.txt	Pinned direct dependencies

File	Responsibility
.streamlit/config.toml	Theme telemetry and headless configuration
run_streamlit.bat	Windows setup train if absent and launch
tests/test_training.py	Split artifact and Streamlit behavior checks
tests/test_detector.py	Original detector checks

The batch launcher changes to its own directory, creates .venv if environment Python is missing, installs requirements, trains only if the artifact is absent, and starts Streamlit. Installation or training failure exits. Source-data changes alone do not force retraining. An incomplete environment without pip may require repair.

The repository excludes .venv, bytecode, ZIP archives, temporary server logs, document QA images, and secrets.toml. No account credential is required by inference. JSON fields include model, data_source, limitations, split_counts, test_families, candidates, selected_C, threshold, versions, dataset_sha256, elapsed time, detector metrics, errors, and family_results.

19 Preserved Naive Bayes baseline

grams creates a Counter of character bigrams and trigrams from normalized text padded with a space at each end. NaiveBayes.fit maintains feature counts and document counts by class and the union vocabulary. It uses raw counts rather than TF-IDF.

Feature likelihood is (class feature count + 1)/(total class feature count + vocabulary size), giving Laplace smoothing. predict starts with log class priors and adds each known feature count times its log likelihood. Unknown features are skipped. Logarithms avoid multiplication of many tiny probabilities.

The class-score difference is clipped from -700 to 700 before exponential conversion to reduce overflow. Influential grams rank likelihood differences. Overlapping grams violate strict conditional independence, affecting score interpretation. This baseline is retained for education, not used in current Streamlit inference.

The original app trains NB at import on legacy rows. GET / returns HTML, GET /api/metrics returns legacy metrics, and POST /api/detect accepts JSON text and returns legacy predictions plus checks. Body size is capped at 20,000 bytes; text must be nonblank and at most 4,000 characters. ThreadingHTTPServer binds localhost on port 8765. It is not the Cloud entrypoint.

20 Verification and reproducibility

Ten unittest checks pass locally. Four original checks cover benign quotes, dataset separation, encoding and comments, and NB learning. Six new checks cover corpus validation, normalized cross-split duplicate rejection, conflicting-label rejection, saved-model repeatability, validation-based candidate selection, and Streamlit state behavior.

AppTest confirms tab names, runs ML, checks shared results, changes to normal username, checks stale analysis is removed, and compares using that new input. Manual browser inspection confirms feature chips, score bar, threshold, and verdict rendering. AppTest does not validate all animation frames, breakpoints, or SQL dialects.

Reproduce by creating a Python virtual environment, installing requirements.txt, running train_model.py, running python -m unittest discover -s tests -v, and starting python -m streamlit run streamlit_app.py. An

external CSV is supplied with train_model.py --csv path/to/corpus.csv. Restart Streamlit after retraining because caches retain prior objects.

21 Streamlit Cloud deployment

GitHub hosts source, dependencies, model, and metrics. Community Cloud deploys main with streamlit_app.py as entrypoint and Python 3.12 to match training. The included model loads at startup; training is not rerun for every visitor. No inference secret is required.

config.toml sets primary blue, pale background, white secondary background, dark text, sans-serif font, gatherUsageStats=false, and headless=true. Fixed localhost address and port were removed from shared config so hosting controls network binding. Local Streamlit uses its normal defaults.

Sign-in and GitHub repository access are necessary. A public URL must be tested before declaring deployment successful. Service account terms and repository authorization are distinct from code preparation. Cloud submissions are processed by the hosting service. No custom payload database or explicit application payload log exists, but infrastructure logging is outside the app's control. Use demonstration inputs rather than sensitive production data.

22 Reliability and security limitations

Joblib can execute code while loading an untrusted artifact. Only the trusted bundled file is loaded; there is no model-upload widget. Version compatibility is important, and upgrades require renewed model and UI checks. HTML escaping must remain intact in future animation changes.

The 4,000-character limit limits accidental oversized inputs but is not authentication, rate limiting, or comprehensive abuse protection. There is no persistent audit trail, user authorization, SQL parser, query context, automatic retraining, calibration, or production integration. Model drift and adversarial evasion are not automatically handled.

Missing artifact gives a user instruction; malformed artifacts, dependency mismatches, or missing JSON can fail startup. Inspect logs and reproduce with pinned dependencies rather than inventing fallback metrics. Public demonstration deployment is not certification for security use.

23 Improvements and interpretation

The next step is an independently sourced licensed corpus, reviewed labels, sensible source or family grouping, and a frozen evaluation protocol. Report uncertainty at family level and compare both ML algorithms on identical data. Separate prediction from explanation timing and use repeated controlled timing with hardware recorded.

Calibration and threshold selection require validation data separate from final testing. Broader rules or query-aware parsing may improve behavior but change the scope. Production use requires secure query construction and operational controls. Current evidence supports an ML advantage in this authored benchmark, while rules remain faster and simpler.

24 Glossary and viva answers

A family groups one template's variants. A split separates fitting, selection, and evaluation. Leakage means evaluation information affects fitting or selection. A vocabulary maps character fragments to columns. Sparse vectors retain nonzero entries. A coefficient weights a feature; the intercept sets baseline logit. Regularization limits coefficient growth. A threshold turns a score into a decision.

Why two detectors? Rules illustrate known signatures; ML learns combinations. Why characters? Punctuation and partial syntax matter. Why Logistic Regression? Sparse text efficiency and coefficient explanations. Are the chips the whole decision? No. Is the score calibrated? No. Does Not flagged guarantee safety? No. Is SQL executed? No. Is current ML Naive Bayes? No. Does the benchmark change for each input? No; it is loaded from the frozen test report.

25 Technical references

Project-specific measurements and behavior come from checked source, saved pipeline, corpus, tests, and trained_metrics.json. Official library references explain concepts and do not independently validate our data or security claims.

TF-IDF API https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.TfidfVectorizer.html

Logistic Regression API https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html

Model persistence https://scikit-learn.org/stable/model_persistence.html

Streamlit session state https://docs.streamlit.io/develop/api-reference/caching-and-state/st.session_state

Streamlit iframe https://docs.streamlit.io/develop/api-reference/layout/st.iframe

Streamlit deployment https://docs.streamlit.io/deploy/streamlit-community-cloud/deploy-your-app/deploy

OWASP SQL injection prevention https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html

26 Exact rule expressions and implementation notes

Boolean condition uses the expression \b(?:or|and)\s+(?:\d+\s*=\s*\d+|'[^']*'\s*=\s*'[^']*'). The \b boundary anchors a keyword edge; \d+ accepts one or more digits; [^']* accepts nonquote characters. It does not evaluate the equality or recognize a complete quoted-string grammar.

UNION query uses \bunion\s+(?:all\s+)?select\b. The question mark makes the ALL group optional. Stacked command uses ;\s*(?:drop|delete|update|insert|alter|exec)\b. re.search finds these anywhere in the normalized string rather than requiring the entire string to match.

Delay function uses \b(?:sleep|benchmark|pg_sleep)\s*\(|\bwaitfor\s+delay\b. Escaped opening parentheses mean literal punctuation. Comment after quote uses ['\"]\s*(?:--|#|/*). Metadata extraction uses \binformation_schema\b|\bload_file\s*\(. These strings are fixed Python raw-string patterns in detector.py.

The UI CSS sets a maximum content width of 1,200 pixels. Tab buttons use 48-pixel minimum height and action buttons 44 pixels. White cards, blue rule accents, purple ML accents, red suspicious labels, and green unflagged labels create hierarchy. Animation keyframes are pulse, packet, enter, and fill. Rule evidence uses a 0.07-second stagger; feature chips use 0.1 seconds. The verdict starts after the main delay plus 0.6 seconds. The mobile animation breakpoint is 500 pixels.

HTML displays at most the first 200 characters of the normalized payload in its visual card, while classification uses the entire input up to 4,000 characters. Visible feature spaces become the symbol for a space, and contribution text is formatted to two decimal places. ML score displays one decimal percent while returned score uses six decimal places. Formatting can therefore round a score near the threshold; the Boolean decision always uses the underlying unrounded value.

The current JSON is read-only in the app; benchmark download serializes it with indentation and application/json MIME type. dataframe formatting shows one decimal percentage and hides its index. Confusion values are integers, per-family accuracies are fractions, and elapsed values are floating-point seconds or milliseconds as labeled. This distinction prevents confusion between 0.9635 and 96.35 percent.